Name: Sapthami Upadhya
Section: CSE A
Roll No.: 15
Reg. No.: 230905090

Lab 2

1. Write a MPI program using synchronous send. The sender process sends a word to the receiver. The second process receives the word, toggles each letter of the word and sends it back to the first process. Both processes use synchronous send operations.

- **Code:**

```
//synch send: sender sends word, receiver receives, toggles each letter,
sends back
//Sapthami Upadhya
//16 Jan 2026
#include<mpi.h>
#include<stdio.h>
#include<ctype.h>
#include<string.h>
void toggle(char* word){
    for(int i = 0; i < strlen(word); i++){
        if(islower(word[i])) word[i] = toupper(word[i]);
        else if(isupper(word[i])) word[i] = tolower(word[i]);
    }
}
int main(int argc, char* argv[]){
    int rank, size;
    char word[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if(rank == 0){
        printf("Enter a word to send: ");
        scanf("%s", word);
        MPI_Ssend(word, strlen(word)+1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Process 0: Sent the word '%s'\n", word);
        fflush(stdout);
        MPI_Recv(word, 100, MPI_CHAR, 1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process 0: Received the toggled word '%s'\n", word);
        fflush(stdout);
    }
    else{
        MPI_Recv(word, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process 1: Received the word '%s'\n", word);
        fflush(stdout);
        toggle(word);
        MPI_Ssend(word, strlen(word)+1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
```

```
        printf("Process 1: Sent the toggled word '%s'\n", word);
        fflush(stdout);
    }
    MPI_Finalize();
    return 0;
}
```

- **Output:**

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpicc ex1.c -o 1to1sync
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpirun -n 2 ./1to1sync
Enter a word to send: hElLo
Process 0: Sent the word 'hElLo'
Process 1: Received the word 'hElLo'
Process 0: Received the toggled word 'HeLlO'
Process 1: Sent the toggled word 'HeLlO'
```

2. Write a MPI program where the master process (process 0) sends a number to each of the slaves and the slave processes receive the number and prints it. Use standard send.

- **Code:**

```
//std send: sender sends number, receivers receive and print the number
//Sapthami Upadhya
//16 Jan 2026
#include<mpi.h>
#include<stdio.h>
int main(int argc, char* argv[]){
    int rank, size, num1;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0){
        printf("Enter the number to send: ");
        scanf("%d", &num1);
        for(int i = 1; i < size; i++){
            MPI_Send(&num1, 1, MPI_INT, i, 0, MPI_COMM_WORLD);
            printf("Process 0: Sent the number %d to process %d\n", num1,
i);
        }
    }
    else{
        int num2;
        MPI_Recv(&num2, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process %d: Received the number %d\n", rank, num2);
    }
    MPI_Finalize();
    return 0;
}
```

- **Output:**

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpicc ex2.c -o 1tomanystd
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpirun -n 5 ./1tomanystd
Enter the number to send: 777
Process 0: Sent the number 777 to process 1
Process 0: Sent the number 777 to process 2
Process 0: Sent the number 777 to process 3
Process 0: Sent the number 777 to process 4
Process 1: Received the number 777
Process 2: Received the number 777
Process 3: Received the number 777
Process 4: Received the number 777
```

3. Write a MPI program to read N elements of the array in the root process (process 0) where N is equal to the total number of processes. The root process sends one value to each of the slaves. Let even ranked process finds square of the received element and odd ranked process finds cube of received element. Use Buffered send.

- **Code:**

```c
//buf send: sender reads an array, sends one element each to receivers,
//even receiver finds square of the number, odd receiver finds cube of the
number
//Sapthami Upadhya
//16 Jan 2026
#include<mpi.h>
#include<stdio.h>
#include<stdlib.h>
int main(int argc, char* argv[]){
    int rank, size;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    int data[5];
    int buf_size = MPI_BSEND_OVERHEAD + sizeof(int);
    void* buf = malloc(buf_size);
    MPI_Buffer_attach(buf, buf_size);

    if(rank == 0){
        printf("Enter the %d array elements: ", size);
        fflush(stdout);

        for(int i = 0; i < size; i++)
            scanf("%d", &data[i]);
        for(int i = 1; i < size; i++){
            MPI_Bsend(&data[i], 1, MPI_INT, i, 0, MPI_COMM_WORLD);
            printf("Process 0: Sent %d to process %d\n", data[i], i);
            fflush(stdout);
        }
        printf("Process 0: Square of %d = %d\n", data[0], data[0]*data[0]);
        fflush(stdout);
```

```
    }
    else{
        int num;
        MPI_Recv(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process %d: Received %d\n", rank, num);
        fflush(stdout);

        if(rank % 2)//odd
            printf("Process %d: Cube of %d = %d\n", rank, num,
num*num*num);
        else
            printf("Process %d: Square of %d = %d\n", rank, num, num*num);
        fflush(stdout);
    }

    MPI_Buffer_detach(&buf, buf_size);
    free(buf);
    MPI_Finalize();
    return 0;
}
```

- **Output:**

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpicc ex3.c -o 1tomanybuf
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpirun -n 5 ./1tomanybuf
Enter the 5 array elements: 2 3 2 1 6
Process 0: Sent 3 to process 1
Process 0: Sent 2 to process 2
Process 0: Sent 1 to process 3
Process 0: Sent 6 to process 4
Process 0: Square of 2 = 4
Process 1: Received 3
Process 1: Cube of 3 = 27
Process 2: Received 2
Process 2: Square of 2 = 4
Process 3: Received 1
Process 3: Cube of 1 = 1
Process 4: Received 6
Process 4: Square of 6 = 36
```

4. Write a MPI program to read an integer value in the root process. Root process sends this value to Processl, Process 1 sends this value to Process2 and so on. Last process sends the value back to root process. When sending the value each process will first increment the received value by one. Write the program using point to point communication routines.

- **Code:**

```
//std send: root reads n, sends n+1 to process 1, process sends n+2 to
process 2 and so on,
//process p sends n+p to root
```

```c
//Sapthami Upadhya
//16 Jan 2026
#include<mpi.h>
#include<stdio.h>
int main(int argc, char* argv[]){
    int rank, size, num;
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if(rank == 0){
        printf("Enter a number: ");
        fflush(stdout);
        scanf("%d", &num);
        MPI_Send(&num, 1, MPI_INT, 1, 0, MPI_COMM_WORLD);
        printf("Process 0: Sent %d to process 1\n", num);
        fflush(stdout);
        MPI_Recv(&num, 1, MPI_INT, size-1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process 0: Received %d from process %d\n", num, size-1);
        fflush(stdout);

    }
    else if(rank > 0 && rank < size-1){
        MPI_Recv(&num, 1, MPI_INT, rank-1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process %d: Received %d from process %d\n", rank, num,
rank-1);
        fflush(stdout);
        num++;
        MPI_Send(&num, 1, MPI_INT, rank+1, 0, MPI_COMM_WORLD);
        printf("Process %d: Sent %d to process %d\n", rank, num, rank+1);
        fflush(stdout);
    }
    else{
        MPI_Recv(&num, 1, MPI_INT, rank-1, 0, MPI_COMM_WORLD,
MPI_STATUS_IGNORE);
        printf("Process %d: Received %d from process %d\n", rank, num,
rank-1);
        fflush(stdout);
        num++;
        MPI_Send(&num, 1, MPI_INT, 0, 0, MPI_COMM_WORLD);
        printf("Process %d: Sent %d to process 0\n", rank, num);
        fflush(stdout);

    }
    MPI_Finalize();
    return 0;
}
```

- **Output:**

```
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpicc ex4.c -o 1to1seqstd
STUDENT@MIT-ICT-LAB5-15:~/Desktop/230905090/Lab2$ mpirun -n 5 ./1to1seqstd
Enter a number: 777
Process 0: Sent 777 to process 1
Process 1: Received 777 from process 0
Process 1: Sent 778 to process 2
Process 2: Received 778 from process 1
Process 2: Sent 779 to process 3
Process 3: Received 779 from process 2
Process 3: Sent 780 to process 4
Process 4: Received 780 from process 3
Process 4: Sent 781 to process 0
Process 0: Received 781 from process 4
```